Implementation of Deep Neural Networks

Regularization, Hyperparameters, and Early Stopping

DSC202: DEEP LEARNING

Lab Session

Motivation

- Machine learning: manual feature engineering
- Deep learning enables end-to-end feature learning
- Enabled by:
  - Large datasets
  - GPU acceleration
  - Efficient optimization algorithms

Session Objectives

- Construct a **Multi-Layer Perceptron (MLP)** using PyTorch/TensorFlow.
- Implement **Regularization** to combat overfitting.
- Apply **Early Stopping** based on validation performance.
- Perform **Hyperparameter Tuning** (Learning rate, Batch size, Architecture).
- Analyze results via Loss/Accuracy curves.

Simple Code Example

```
import torch
import torch.nn as nn

# Linear model
linear = nn.Linear(2, 1)

# Non-linear model
non_linear = nn.Sequential(
    nn.Linear(2, 10),
    nn.ReLU(),
    nn.Linear(10, 1)
)
```

- Non-linear models capture complex patterns

Feedforward Neural Network

$$y = f(\mathbf{W}x + b)$$

- Information flows forward only
- Also called Multi-Layer Perceptron (MLP)

From Linear and Logistic Regression to MLPs

- **Linear Regression**:

$$y = \mathbf{w}^\top \mathbf{x} + b$$

- **Logistic Regression**:

$$y = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

- Both can be viewed as **single-layer neural networks**
- No hidden layers $\Rightarrow$ limited expressive power
- MLPs extend this by stacking multiple non-linear layers

Linear and Logistic Regression as MLPs

```
# Linear Regression as a Neural Network
linear_model = nn.Linear(<N_feat>, 1) # e.g. 2

# Logistic Regression as a Neural Network
logistic_model = nn.Sequential(
    nn.Linear(<N_feat>, 1),
    nn.Sigmoid()
)

# With hidden layer
mlp_model = nn.Sequential(
    nn.Linear(2, 16),
    nn.ReLU(),
    nn.Linear(16, 1),
    nn.Sigmoid()
)
```

Building an MLP

```
class MLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = torch.relu(x)
        x = self.fc2(x)
        return x


X = torch.rand(100, 15)
# 100 samples, each with 15 random features
y = torch.randint(0, 2, (100, 1)).float()
# Binary labels (0 or 1), 100 samples
```

Activation Functions

- Introduce non-linearity
- Enable learning complex decision boundaries
- Common activations:
  - ReLU
  - Sigmoid
  - Tanh
  - Softmax

Activation Functions

```
x = torch.randn(5)

relu_out = torch.relu(x)
sigmoid_out = torch.sigmoid(x)
tanh_out = torch.tanh(x)

print(relu_out, sigmoid_out, tanh_out)
```

Why Regularization?

- Deep models can overfit
- Perform well on training but poorly on test data

L2 Regularization (Weight Decay)

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4  # L2 regularization
)
```

Dropout Regularization

```
model = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 10)
)
```

- Randomly disables neurons during training

Early Stopping (Concept)

- Monitor validation loss
- Stop training when validation loss increases
- Prevents overfitting

What are Hyperparameters?

- Learning rate
- Batch size
- Number of layers
- Number of neurons

Hyperparameter Tuning (Code Example)

```
learning_rates = [0.1, 0.01, 0.001]

for lr in learning_rates:
    optimizer = torch.optim.Adam(
        model.parameters(), lr=lr)
    print(f"Training with lr={lr}")
```

Training Loop (Complete Example)

```
criterion = nn.CrossEntropyLoss()

for epoch in range(epochs):
    for x, y in dataloader:
        optimizer.zero_grad()
        outputs = model(x)
        loss = criterion(outputs, y)
        loss.backward()
        optimizer.step()
```

Exercise 1: Model Architecture

- Add one more hidden layer to the MLP
- Experiment with different hidden sizes
- Observe training loss

Exercise 2: Activation Functions

- Replace ReLU with Tanh
- Compare convergence speed
- Report accuracy differences

Exercise 3: Regularization

- Train with and without dropout
- Compare training vs validation loss

Exercise 4: Hyperparameter Tuning

- Tune learning rate and batch size
- Identify best configuration

Bonus Exercise

- Implement early stopping
- Plot loss curves

Optimization

- **Loss Function:** *Cross-Entropy Loss* is standard for classification.
- **Optimizer:** *Adam* is generally the preferred starting point due to adaptive learning rates.
- **Weight Decay:** $L2$ regularization can be added inside the optimizer settings.

Post-Implementation Analysis

To complete the practical, your report should include:

1. **Learning Curves:** Plot Training vs. Validation Loss.
2. **Accuracy Metrics:** Final Top-1 accuracy on the test set.
3. **Ablation Study:** Compare performance "With Dropout" vs. "Without Dropout."
4. **Conclusion:** Which hyperparameter configuration yielded the best generalization?

Motivation for CNNs

- Fully Connected Networks (MLPs):
  - Ignore spatial structure
  - Large number of parameters
  - Poor scalability for images
- CNNs:
  - Designed for grid-like data (images, sequences)
  - Use local connectivity
  - Share parameters (kernels)
  - Learn hierarchical spatial features

Images as Numerical Data

- Grayscale image: $H \times W \times 1$
- RGB image: $H \times W \times 3$
- Each pixel stores intensity values.
- CNNs operate directly on these pixel grids.

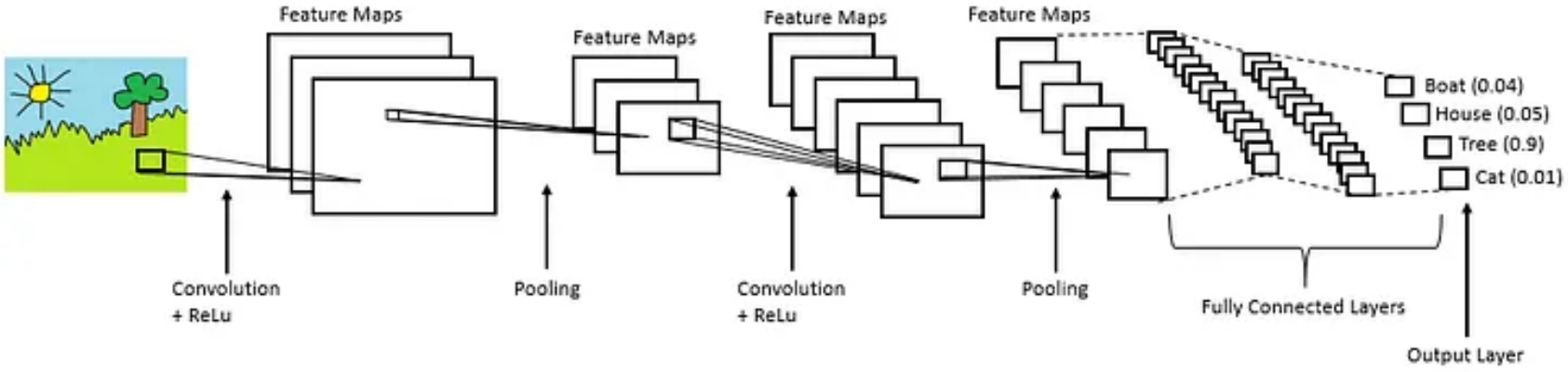

Figure: Complete CNN Architecture

What is Convolution?

- A small learnable filter (kernel) slides over input.
- Performs element-wise multiplication and summation.
- Produces a feature map.

$$Output(i, j) = \sum_m \sum_n Input(i + m, j + n) \cdot Kernel(m, n)$$

- Learns edges, textures, patterns automatically.

Channels and Filters

- Each filter spans all input channels.
- One filter produces one output feature map.
- Number of filters = number of output channels.
- Multiple filters learn different features.

Stride

- Stride = step size of kernel movement.
- Stride = 1 $\rightarrow$ detailed feature extraction.
- Stride = 2 $\rightarrow$ reduced spatial dimension.

Output Size:

$$\frac{(N - F + 2P)}{S} + 1$$

Where:

- N = input size
- F = filter size
- P = padding
- S = stride

Padding

- Without padding: spatial size reduces rapidly.
- Padding adds border pixels (usually zeros).
- Helps preserve edge information.

Pooling Layers

- Downsamples spatial dimensions.
- No learnable parameters.

Types:

- Max Pooling
- Average Pooling

Example:

$$28 \times 28 \rightarrow 14 \times 14$$

Why Pooling?

- Reduces computation.
- Controls overfitting.
- Provides translation invariance.

Stacking Convolutional Layers

Typical CNN Pipeline:

$$\begin{gathered}\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \\ \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \\ \text{Flatten} \rightarrow \text{Fully Connected} \rightarrow \text{Softmax}\end{gathered}$$

- Early layers: edges
- Middle layers: textures
- Deep layers: object-level features

CNN vs MLP

Feature	MLP	CNN
Connections	Fully connected	Local connectivity
Parameters	Very large	Reduced (weight sharing)
Spatial awareness	No	Yes
Best suited for	Tabular data	Images/Text

Simple CNN in PyTorch

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1,16,3,padding=1)
        self.pool = nn.MaxPool2d(2,2)
        self.conv2 = nn.Conv2d(16,32,3,padding=1)
        self.fc1 = nn.Linear(32*7*7,128)
        self.fc2 = nn.Linear(128,10)

    def forward(self,x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)
```

Applications: Image Classification

- Handwritten digit recognition
- Object classification
- Medical image diagnosis
- Face recognition

Applications: Text Classification

- Apply 1D convolution on word embeddings.
- Learns local phrase patterns.
- Used in:
  - Sentiment analysis
  - Spam detection
  - Topic classification

Student Exercises

1. Change stride and observe output dimension.
2. Compare Max Pooling vs Average Pooling.
3. Increase number of filters and analyze performance.
4. Apply CNN to a text dataset.

Key Notes

- CNNs exploit spatial structure.
- Convolution reduces parameters via weight sharing.
- Pooling improves robustness.
- Effective for both images and text.